

Multi-Agent Reinforcement Learning

Assignment 1

Sambit Sahoo (22277)

Please find the source code at GitHub or Drive Link

Q1- Differential-Drive Robot with Orientation-Dependent Planning

(a) The MDP formulation of the problem is shown below.

Question-1

② MDP = Markov Decision Process

$M = (S, A, P, R, \gamma)$

State space S as mentioned in quest.
 $s = (x, y, \theta)$
 $x, y \in \{0, 1, \dots, 9\}$ (10×10 grid)
 $\theta \in \{0, \frac{\pi}{2}, \pi, \frac{3\pi}{2}\}$

Action space $A = \{\text{Forward, Turn left, Turn right}\}$

Transition Model: $P(s'|s, a)$

Turn actions:
 a) left $(x, y, \theta) \rightarrow (x, y, \theta + \frac{\pi}{2})$
 b) right $(x, y, \theta) \rightarrow (x, y, \theta - \frac{\pi}{2})$

Forward action:
 $P(s'|s, \text{Forward}) = \begin{cases} 0.8 & \text{forward} \\ 0.1 & \text{left slip} \\ 0.1 & \text{right slip} \end{cases}$

Discrete vectors based on orientations

$\delta_f(s) = (\Delta x_f, \Delta y_f)$ Forward
 $\delta_L(s) = (\Delta x_L, \Delta y_L)$ Left slip
 $\delta_r(s) = (\Delta x_r, \Delta y_r)$ Right slip

θ	δ_f	δ_L	δ_r
0 (E)	(1, 0)	(0, 1)	(0, -1)
$\frac{\pi}{2}$ (N)	(0, 1)	(1, 0)	(-1, 0)
π (W)	(-1, 0)	(0, -1)	(0, 1)
$\frac{3\pi}{2}$ (S)	(0, -1)	(-1, 0)	(1, 0)

Diagram of a coordinate system with North (N), East (E), South (S), and West (W) directions.

$P(s' | (x, y, \theta), \text{Forward}) = \begin{cases} 0.8 & \text{if } s' = (x + \Delta x_f, y + \Delta y_f, \theta) \\ 0.1 & \text{if } s' = (x + \Delta x_L, y + \Delta y_L, \theta) \\ 0.1 & \text{if } s' = (x + \Delta x_r, y + \Delta y_r, \theta) \\ 0 & \text{otherwise} \end{cases}$

Reward:
 $r_{\text{boundary / forbidden}} = -100$
 $r_{\text{valid}} = -1$
 $r_{\text{goal}} = +50$

$P(r | s, a) = \begin{cases} 1, & r = -100 \text{ if } s' \text{ is collision} \\ 1, & r = +50 \text{ if } s' \text{ is goal} \\ 1, & r = -1 \text{ otherwise} \\ 0, & \text{otherwise for all other } r \end{cases}$

Discount factor
 $\gamma = 0.95$ (Assumption)

(b), (c) Value iteration and Policy iteration for 3 iterations.

Value iteration for 3 iterations.

First iteration - $v(s) = 0$ for all s

$s = (0, 0, 0)$ $\theta \rightarrow \text{East}$

$q(s, L) = -1 + \gamma v(0, 0, \pi/2) = -1$
 $q(s, R) = -1 + \gamma v(0, 0, \pi/2) = -1$
 $q(s, F) = [0.1(-100) + 0.1(-1) + 0.8(-1)] + \gamma[0.1v(0, 0, 0) + 0.1v(0, 1, 0) + 0.8v(1, 0, 0)]$
 $= [-10 - 0.1 - 0.8] + 0.9[0 + 0 + 0]$
 $= -10.9$

$v_1(0, 0, 0) = -10.9$ $\pi_1((0, 0, 0)) = L$

$s = (8, 2, 1) \rightarrow$ Near obstacle

$q(s, L) = -1 + \gamma 0 = -1$
 $q(s, R) = -1 + \gamma 0 = -1$
 $q(s, F) = 0.8(-100) + 0.1(-1) + 0.1(-1) = -80.2$ { as $(3, 3) \rightarrow$ obstacle
 $(2, 2) \rightarrow$ valid
 $(4, 2) \rightarrow$ valid }

$V_1(8, 2, 1) = -1$, $\pi_1(8, 2, 1) = L$

$s = (8, 9, 1)$

$q(s, L) = -1 + \gamma 0 = -1$
 $q(s, R) = -1 + \gamma 0 = -1$
 $q(s, F) = 0.8(-100) + 0.1(-1) + 0.1(50) = -75.1$ { as $(8, 9) \rightarrow$ boundary
 $(7, 9) \rightarrow$ valid
 $(9, 9) \rightarrow$ goal }

$V_1(8, 9, 1) = -1$, $\pi_1(8, 9, 1) = R$

Iteration-2

From code $V_1(0, 0, 1) = -1$ (terminal)
 $V_1(0, 0, 2) = -1$
 $V_1(1, 0, 0) = -1$
 $V_1(0, 1, 0) = -1$
 $V_1(\text{collision}) = 0$

$s = (0, 0, 0)$

$q(s, L) = -1 + 0.95(-1) = -1.95$
 $q(s, R) = -1 + 0.95(-1) = -1.95$
 $q(s, F) = 0.8(-1 + 0.95(-1)) + 0.1(-100 + 0.95(0)) + 0.1(-1.95) = -11.755$

$V_2(0, 0, 0) = -1.95$, $\pi_2(0, 0, 0) = L$

$s = (3, 2, 1)$

From code $V_1(3, 2, 1) = -1$
 $V_1(3, 2, 0) = -1$
 $V_1(2, 2, 1) = -1$
 $V_1(4, 2, 1) = -1$
 $V_1(\text{collision}) = 0$

$q(s, L) = -1 + 0.95(-1) = -1.95$
 $q(s, R) = -1 + 0.95(-1) = -1.95$
 $q(s, F) = 0.8(-100 + 0.95(-1)) + 0.1(-1 + 0.95(-1)) + 0.1(-1.95) = -80.39$

$V_2(3, 2, 1) = -1.95$, $\pi_2(3, 2, 1) = L$

Iteration-3

From code $V_2(0, 0, 1) = -1.95$
 $V_2(0, 0, 2) = -1.95$
 $V_2(1, 0, 0) = -1.95$
 $V_2(0, 1, 0) = -1.95$
 $V_2(\text{collision}) = 0$

$s = (0, 0, 0)$

$q(s, L) = -1 + 0.95(-1.95) = -2.8525$
 $q(s, R) = -1 + 0.95(-1.95) = -2.8525$
 $q(s, F) = 0.8(-1 + 0.95(-1.95)) + 0.1(-1 + 0.95(-1.95)) + 0.1(-100 + 0.95(0)) = -12.567$

$V_3(0, 0, 0) = -2.8525$, $\pi_3(0, 0, 0) = L$

$s = (3, 2, 1)$

From code $V_2(3, 2, 2) = -1.95$
 $V_2(3, 2, 0) = -1.95$
 $V_2(2, 2, 1) = -1.95$
 $V_2(4, 2, 1) = -1.95$
 $V_2(\text{collision}) = 0$

$q(s, L) = -1 + 0.95(-1.95) = -2.8525$
 $q(s, R) = -1 + 0.95(-1.95) = -2.8525$
 $q(s, F) = 0.8(-100 + 0.95(-2.8525)) + 0.1(-1 + 0.95(-2.8525)) + 0.1(-2.8525) = -80.67$

$V_3(3, 2, 1) = -2.8525$, $\pi_3(3, 2, 1) = L$

$s = (8, 9, 1)$

From code $V_2(8, 9, 2) = -1.95$
 $V_2(8, 9, 0) = 27.405$
 $V_2(7, 9, 1) = -1.95$
 $V_2(\text{collision}) = 0$
 $V_2(\text{terminal goal}) = 50$

$q(s, L) = -1 + 0.95(-1.95) = -2.8525$
 $q(s, R) = -1 + 0.95(27.405) = 27.81475$
 $q(s, F) = 0.8(-100) + 0.1(-1 + 0.95(-1.95)) + 0.1(50) = -75.28525$

$V_3(8, 9, 1) = -2.8525$, $\pi_3(8, 9, 1) = R$

Value Function after iteration 1:

$V_1(0, 0, 0) = -1.000$
 $V_1(3, 2, 1) = -1.000$
 $V_1(8, 9, 1) = -1.000$

Value Function after iteration 2:

$V_2(0, 0, 0) = -1.950$
 $V_2(3, 2, 1) = -1.950$
 $V_2(8, 9, 1) = 27.405$

Value Function after iteration 3:

$V_3(0, 0, 0) = -2.853$
 $V_3(3, 2, 1) = -2.853$
 $V_3(8, 9, 1) = 27.315$

Sample optimal actions:

$\pi((0, 0, 0)) = L$
 $\pi((3, 2, 1)) = L$
 $\pi((8, 9, 1)) = R$

Policy Iteration 1

$\pi_1((0, 0, 0)) = L \mid V = -1.000$
 $\pi_1((3, 2, 1)) = L \mid V = -1.000$
 $\pi_1((8, 9, 1)) = R \mid V = -1.000$

Policy Iteration 2

$\pi_2((0, 0, 0)) = L \mid V = -1.950$
 $\pi_2((3, 2, 1)) = L \mid V = -1.950$
 $\pi_2((8, 9, 1)) = R \mid V = 27.405$

Policy Iteration 3

$\pi_3((0, 0, 0)) = L \mid V = -2.853$
 $\pi_3((3, 2, 1)) = L \mid V = -2.853$
 $\pi_3((8, 9, 1)) = R \mid V = 27.315$

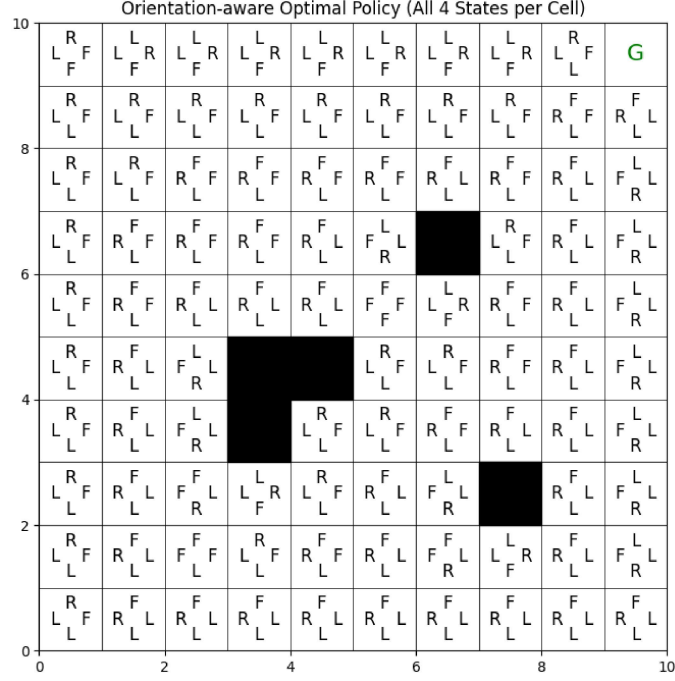
Initial Random Policy:

$\pi_0((0, 0, 0)) = L$
 $\pi_0((3, 2, 1)) = L$
 $\pi_0((8, 9, 1)) = R$

(d) **Comparison Value iteration vs Policy iteration**

Method	Iterations	Runtime (s)	Peak Memory (KB)
Value Iteration	95	0.3489	55.21
Policy Iteration	10	1.2748	53.69

(e) Optimal policy



(f) Step reward, Goal Reward , Collision Reward 1(-1, 50, -100), 2(-1, 50, -10), 3(-15, 50, -100)

Table 1: Comparison of Value Iteration and Policy Iteration under Different Rewards

Method	Metric	Reward1	Reward2	Reward3
Value Iteration	Iterations	95	52	30
	Runtime (s)	0.348878	0.198646	0.138658
	Memory (KB)	55.213867	58.240234	57.888672
Policy Iteration	Iterations	10	10	8
	Runtime (s)	1.274766	0.584009	0.599789
	Memory (KB)	53.687500	53.406250	53.171875

- (g)
- **Policy Iteration** converges in fewer iterations.
 - **Value Iteration** is computationally cheaper per iteration .
 - **Monte Carlo** is model-free but requires many episodes to stabilize.

Method	Iterations / Episodes	Runtime (s)
Value Iteration	95	0.3489
Policy Iteration	10	1.2748
Monte Carlo	5000	0.5208

Q2- Battery-Aware Robot Navigation

- (a) The MDP formulation of the problem is shown below. Assumptions include moves as Up, Down, Right, Left and charging to full capacity at charging station. (Others in the formulation)

Question-02:

② MDP: (S, A, P, R, γ)

State space = $\{(x, y, b) \mid x \in \{0, 1, \dots, 9\}, y \in \{0, \dots, 9\}\}$
 and S_F = failure state
 $|S| = 100 \times 10 + 1 = 1001$

Action space = $\{Up, Down, Left, Right, Recharge\}$
 $a \in \{Up, Down, Left, Right, Recharge\}$

Transition Model: $P(s' | s, a) \rightarrow 3 \text{ cases}$

① Movement Action: $a \in \{U, D, L, R\}$
 $(x', y') = f_a(x, y)$

Case-A - Valid Move, $b > 1$
 $P((x', y', b-1) | (x, y, b), a) = 1$

Case-B - Valid Move, $b = 1$
 $\star$ If $(x', y') \in C$ $C \rightarrow$ charging station
 $P((x', y', 0) | (x, y, 1), a) = 1$

$\star$ If $(x', y') \notin C$
 $P(s_F | (x, y, 1), a) = 1$

Case-C Invalid Move (boundary) or
~~Recharge~~
 $P((x, y, b-1) | (x, y, b), a) = 1$
 Invalid staying in same state but consume battery

② Recharge Action

Case-A Charging station
 $P((x, y, 9) | (x, y, b), Recharge) = 1$

Case-B Not at charging station (self loop)
 $P((x, y, b) | (x, y, b), Recharge) = 1$

③ Failure State
~~Case-C~~ $P(s_F | s_F, a) = 1$

Reward
 $R(s, a, s') = \begin{cases} +100 & \text{goal} \\ -100 & \text{failure} \\ -1 & \text{move} \\ -2 & \text{recharge} \end{cases}$ (given in ques)

$\gamma \in \{0.99, 0.9, 0.7\}$

- (b) First I created the environment as defined in the formulation, specified the charging stations, actions, valid states reward structure, etc. `get_transitions_battery()` that returns the list of (prob., state, rewards) for each (state, action). `run_value_iteration_battery()` converged values, optimal policy, iterations until delta tol (or max_iters was reached). Similarly, `run_policy_iteration_battery()` for policy iteration (policy evaluation and then improvement).

Value iteration converges at 24 iterations.

Policy iteration converges at 11 iterations.

(c), (d) Here i changed the value of $B = 6$ for better result comparison. Also i simulated the path for optimal policy of the robot starting at $(0,0,6)$ instead of showing policy because difference is more clear here

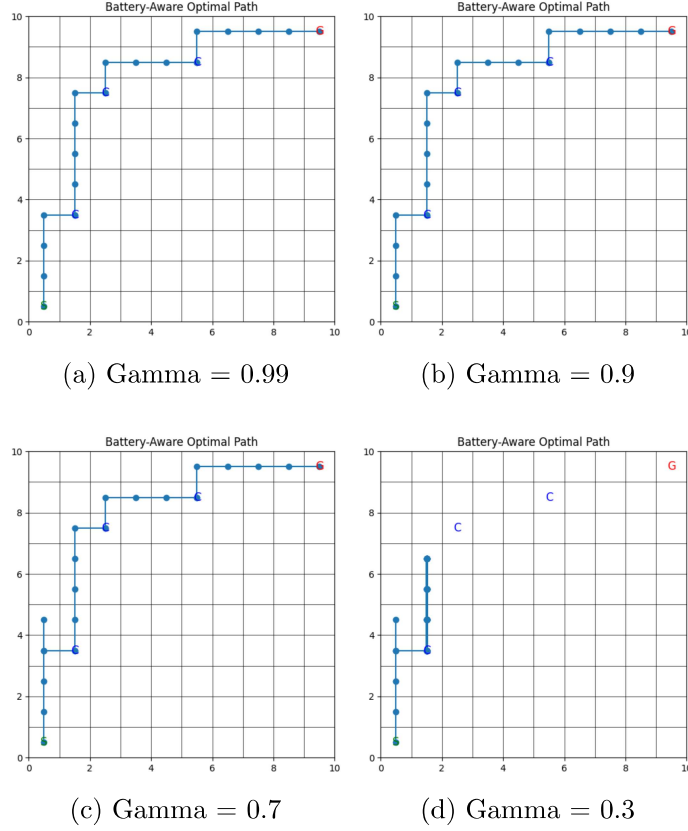


Figure 1: Results for gamma values

- (e) As the discount factor decreases, the robot becomes short sighted, so it tries to consider less about future rewards and more about immediate rewards (as we can see from 0.7 and 0.3) so, avoids long detours to charging stations and may fail to reach the goal because the future reward is heavily discounted.
- (f) Comparison of value iteration policy iteration.

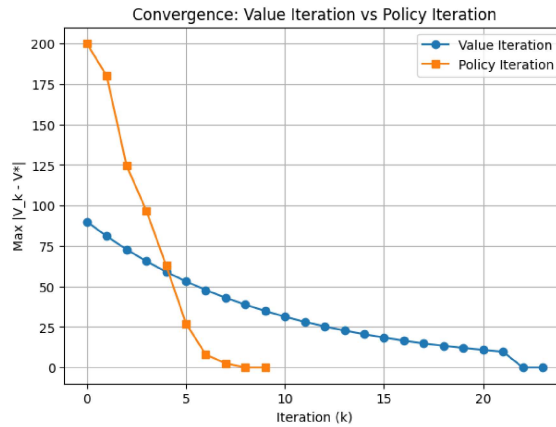


Table 2: Final Comparison (Battery Case)

Method	Iterations / Episodes
Value Iteration	24
Policy Iteration	13
Monte Carlo	8000

Q3- Risk-Sensitive Robot Navigation Near Hazards

- (a) MDP formulation. Assumptions include moves as Up, Down, Right, Left. (Others in the formulation)

Ques-03: MDP formulation

(S, A, P, R, γ)

State Space: $S = (x, y, h) \cup S_F$
 $x, y \in \{0, \dots, 9\}$
 $h = \min_{(x_n, y_n) \in H} (|x - x_n| + |y - y_n|)$
 $\hookrightarrow$ hazardous cell
 h = Manhattan Distance (Assume)
 S_F = Failure (Catastrophic failure)
Terminal States = Goal states $\cup S_F$

Action Space: $A(s) = \{\text{Up, Down, Left, Right}\}$

Transition Probability $P(s'|s, a)$
 s_{int} = Intended next state
 $s_{\text{slip1}}, s_{\text{slip2}}$ = lateral neighboring states
 S_F = Failure
 $P_h = P_{\text{slip}}(h)$

Case 1: Safe Region ($h > 1$)

$$P(s'|s, a) = \begin{cases} 1 - P_h, & s' = s_{\text{int}} \\ P_h/2, & s' = s_{\text{slip1}} \\ P_h/2, & s' = s_{\text{slip2}} \\ 0, & \text{otherwise} \end{cases}$$

Case-2: Near hazard ($h \leq 1$)

$$P(s'|s, a) = \begin{cases} 1 - P_h, & s' = s_{\text{int}} \\ P_h, & s' = S_F \\ 0, & \text{otherwise} \end{cases}$$

Boundary Condition
 $s_{\text{int}} = s$ if move invalid

Reward:

$$R(s', a, s) = \begin{cases} +50, & \text{goal} \\ -200, & \text{catastrophic failure} \\ -1, & \text{step cost} \end{cases}$$

Discount factor:
 $\gamma = 0.9$

Assumption
- Different slip probabilities for different h .

- (d) To study the effect of increased slip probability near hazardous regions, I progressively increased the probability of catastrophic failure as a function of proximity (Manhattan distance h).

When slip probability was non-zero only inside hazard cells (Case 1), the optimal policy resembled the shortest path and did not maintain any safety buffer.

As slip probability was introduced for states with $h = 1$ (Case 2), the policy began to deviate slightly from the hazard boundary.

With further increases (Case 3), a clear safety margin emerged, and the robot avoided states with small h values.

In the highest slip setting (Case 4), the policy became strongly risk-averse and completely avoided regions near the hazard, even if it resulted in a longer path to the goal.

This demonstrates that increasing slip probability increases expected catastrophic cost, causing the optimal policy to trade path length for safety

- (b),(c) The optimal policy is shown below in (d) image. Risk taking regions (where slip probability is highest) is shown as red arrows

Slip Probability Settings

The following proximity-dependent slip probabilities were used to study risk sensitivity:

- **Case 1 (Minimal Risk):**

$$p(h = 0) = 0.3, \quad p(h \geq 1) = 0$$

- **Case 2 (Low Risk Near Hazard):**

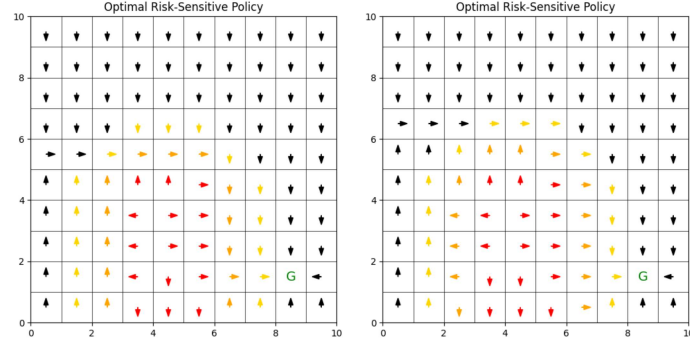
$$p(h = 1) = 0.1, \quad p(h = 0) = 0.4, \quad p(h \geq 2) = 0$$

- **Case 3 (Moderate Risk Gradient):**

$$p(h = 3) = 0.05, \quad p(h = 2) = 0.1, \quad p(h = 1) = 0.2, \quad p(h = 0) = 0.5$$

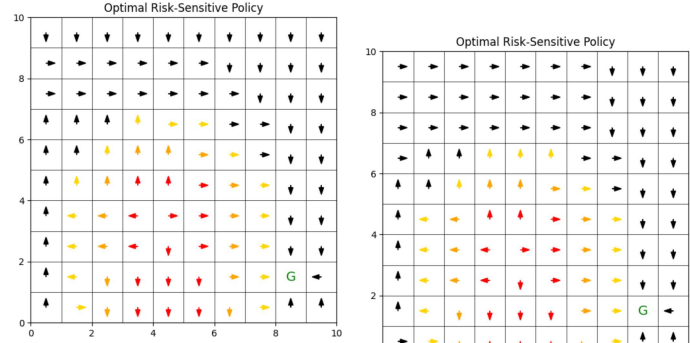
- **Case 4 (High Risk Gradient):**

$$p(h \geq 4) = 0.05, \quad p(h = 3) = 0.1, \quad p(h = 2) = 0.2, \quad p(h = 1) = 0.3, \quad p(h = 0) = 0.7$$



(a) Risk only inside hazard

(b) Mild risk near hazard



(c) Moderate proximity-based risk

(d) High risk sensitivity

Figure 2: Red region is the hazardous region

(e) Why Shortest-Path Intuition Fails

In a deterministic grid, the shortest path minimizes total step cost. However, in this stochastic setting, actions near hazardous regions have a non-zero probability of catastrophic failure with reward -200 . The objective is to maximize expected return:

$$V(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V(s')].$$

Even a small slip probability introduces an expected penalty

$$E[\textit{catastrophic_cost}] = p_{\textit{slip}} \times (-200),$$

which can outweigh the benefit of reducing path length. Therefore, a longer but safer path may yield higher expected value than the geometrically shortest path.